Analysis Report

TRAQS – Requirements Traceability and Quality System

CMPE 491 – TED University 2026–2027 Fall Semester

Senior Design Project I

Supervisor: Mert Burkay Çöteli

Jury members: Eren Ulu, Ayşe Yasemin Seydim

Project web page: https://seratlizzeynep.github.io/TRAQS/

Group Members

Zeynep Aybüke Seratlı – 10057197752

Fatma Eylül Karaoğlu – 35476489950

Ege Ersen – 10330190760

Doruk Özata – 10081192686

November 6, 2026

1. Introduction

TRAQS is a web-based requirements management and traceability platform for teams that build safety-critical products in defense, medical and aerospace; this report analyzes the problem it solves and fixes what the system shall do.

1.1 Purpose of the document

This Analysis Report formalizes the Project Specifications Report and the detailed requirements list (17 user, 40 system, 47 software and 8 non-functional requirements). It acts as the contract between the development team and the customer (the course supervisor and jury): it models the problem domain, resolves ambiguities, and examines boundary conditions and exceptional cases. It is the input of the high-level (architectural) design that follows.

1.2 Scope

TRAQS has two parts. The **core platform** covers the main capabilities of IBM DOORS (used as a reference for concepts only): projects, modules and hierarchical objects with never-reused IDs, typed attributes and saved views, typed links and a traceability matrix, suspect links and impact analysis, a change-request workflow with an audit log, immutable baselines, test cases and verification status, Excel/CSV and ReqIF exchange, a REST API, and role-based access control.

The **AI agent** classifies requirements by domain, type and criticality, scores their quality, rewrites them according to the rules of a domain profile (defense or medical), drafts requirements from uploaded documents, suggests traceability links, and analyzes the complete linked chain. It runs only on a locally hosted language model, and every output stays a suggestion until a user accepts it.

Out of scope: control of any physical system, certification under DO-178C or IEC 62304, ReqIF packages (.reqifz), and domain profiles other than defense and medical (these are configuration and can be added later).

1.3 Objectives and success criteria

- Let a team capture, link, change, baseline and verify requirements entirely in a web browser, without client installation.
- Show, for any change, which requirements, design items and test cases are affected.
- Reduce review effort with AI proposals that are explained (reasons, standard reference, confidence) and always approved by a person.
- Keep all project data inside the organization (local model, TLS, project isolation).
- Demonstrate the full flow on one defense and one medical sample project (UR-17): import, AI classification and quality check, change request, baseline, chain analysis. All Must requirements form the minimum for the final demonstration.

1.4 Stakeholders and users

Stakeholder	Interest in TRAQS
Requirements engineer / Author	Writes, links and maintains requirements; uses the AI agent for drafting and rewriting
Reviewer	Reviews change requests, validates (signs off) requirements
Administrator	Manages users, groups, roles, projects and domain profiles
Viewer / Supplier / Partner	Reads requirements; exchanges them through Excel/CSV and ReqIF
Test engineer	Manages test cases and records results (Pass, Fail, Blocked)
External tool	Reads and writes project data through the REST API
Supervisor and jury (customer)	Defines the reference (DOORS), approves scope and evaluates the demonstration

1.5 Overview of the document

Section 2 describes the current way of working. Section 3 presents the proposed system: overview, functional requirements (3.2), non-functional requirements (3.3), pseudo requirements (3.4) and the system models (3.5: scenarios, use cases, class model, dynamic models, user interface). Section 4 is the glossary and Section 5 the list of references.

1.6 Notation and priorities

Requirement IDs are UR (user), SR (system), SW (software) and NFR (non-functional). Priority follows MoSCoW (Must, Should, Could). Verification methods are T = Test, D = Demonstration, I = Inspection. ID numbers follow the original, longer list, so some numbers are unused. Diagrams use UML.

2. Current System

Today, teams that build safety-critical products manage requirements either with generic office files or with a heavyweight commercial tool such as IBM DOORS, and both ways leave gaps that TRAQS targets.

2.1 How requirements are managed today

- **Documents and spreadsheets.** Requirements live in Word or Excel files, are exchanged by e-mail, and are linked to test cases by hand (cell references, copied IDs).
- **Commercial tools (IBM DOORS as the reference).** They provide modules, typed links, suspect links, baselines and ReqIF exchange, but need client installation, licenses and trained administrators, and they have no built-in AI support.
- **Partner exchange.** Suppliers and customers exchange requirements as Excel or ReqIF files; mapping columns and attributes is manual.
- **Quality review.** Engineers read each requirement and check wording rules (“shall”, vague terms, one requirement per sentence, verification method) and domain rules (defense, medical) by hand.

2.2 Problems of the current way of working

#	Problem	Consequence	Addressed by
P1	IDs are reused or lost when files are edited	Broken traceability, ambiguous references	UR-01
P2	Links to tests and other requirements are manual and unchecked	Gaps and orphans are found late	UR-03, UR-09, UR-16
P3	A change does not show what it affects	Missed impact on safety-critical items	UR-04
P4	No controlled change process, locking or audit trail	Unapproved edits, no accountability	UR-05, UR-06
P5	Baselines are copies of files	No safe restore or comparison	UR-06
P6	Tools need installation and licenses	Suppliers and reviewers cannot take part easily	UR-07, NFR-02, NFR-04
P7	Quality and domain checks are manual and inconsistent	Slow reviews, uneven requirement quality	UR-12, UR-13, UR-14
P8	Drafting requirements from documents is slow	Effort spent on routine writing	UR-15
P9	Cloud AI services would expose classified or personal data	Export-control and KVKK/GDPR risk	UR-15, SR-34
P10	No view of the whole chain (user, system, software, tests)	Coverage unknown until late reviews	UR-16

2.3 Conclusion

No single, freely available, browser-based tool combines DOORS-like traceability with a locally hosted AI assistant that keeps the human in charge. TRAQS fills this gap.

3. Proposed System

3.1 Overview

TRAQS is a browser-based, containerized system with five logical parts: a React single-page application, a REST backend (/api/v1) with a relational database, an import/export module (Excel/CSV, ReqIF), a separate AI service that calls only a configurable local language model endpoint, and an analysis service for chain analysis. The requirements are written top-down in three linked levels: user requirements (UR) state what users need, system requirements (SR) state what the system shall do, and software requirements (SW) state what the software components implement.

Item	Count
User / system / software requirements	17 / 40 / 47
Non-functional requirements	8
SR priority (Must / Should / Could)	29 / 7 / 4
SW priority (Must / Should / Could)	34 / 8 / 5
Verification (Test / Demonstration / Inspection)	by SW, see tables

An SR has the priority of its highest-priority SW; where scope was reduced, a SW can have a lower priority than its SR, and the SW priority is binding. A system requirement that serves several user requirements (SR-14, SR-27, SR-33) is defined once and only referenced afterwards.

3.2 Functional requirements

3.2.1 Core system (UR-01 to UR-09)

ID	Requirement	Parent	Priority	Verif.
UR-01	The user shall be able to capture and organize requirements in hierarchical modules.	–	Must	
SR-01	The system shall manage projects, modules and hierarchical, ordered objects (parent/child).	UR-01	Must	
SW-01	Implement the data model and CRUD API for Project, Module and Object (tree with parent and order).	SR-01	Must	T
SW-02	Implement a tree UI with expand/collapse and drag-and-drop reordering.	SR-01	Must	D
SR-02	The system shall give every object a unique ID that is never reused.	UR-01	Must	
SW-03	Generate sequential IDs (e.g. REQ-0001) and keep deleted IDs on a blacklist.	SR-02	Must	T
UR-02	The user shall be able to define custom attributes and filtered views of requirements.	–	Must	
SR-04	The system shall support user-defined, typed attributes (text, enumeration, number, date).	UR-02	Must	
SW-05	Implement an attribute definition screen and typed attribute storage.	SR-04	Must	T
SR-05	The system shall support saved views with filters, columns and sorting.	UR-02	Should	
SW-06	Implement personal and shared saved views (filter, columns, sort).	SR-05	Should	T
UR-03	The user shall be able to link requirements to other requirements, design items and test cases.	–	Must	
SR-06	The system shall support typed links (Satisfies, Derives, Verifies, Refers) within and across modules.	UR-03	Must	
SW-07	Implement a link table (source, target, type) with an API and UI to create and delete links.	SR-06	Must	T
SR-07	The system shall reject links and hierarchies that break the defined link rules.	UR-03	Must	
SW-08	Implement link rule validation from a single configuration file.	SR-07	Must	T
SR-08	The system shall show upstream and downstream traceability and a coverage matrix.	UR-03	Must	
SW-09	Implement a traceability matrix page and an upstream/downstream tree view.	SR-08	Must	D
UR-04	The user shall be able to see which items are affected when a requirement changes.	–	Must	

ID	Requirement	Parent	Priority	Verif.
SR-09	The system shall mark links as suspect when a linked object changes.	UR-04	Must	
SW-10	Set related links to suspect in the same transaction when object text changes.	SR-09	Must	T
SW-11	Implement a logged “clear suspect” action that requires a reason.	SR-09	Must	T
SR-10	The system shall produce an impact analysis for a selected change.	UR-04	Must	
SW-12	Implement a recursive impact query and an impact analysis view.	SR-10	Must	T
UR-05	The user shall be able to propose, review and approve changes through a change management workflow.	–	Must	
SR-11	The system shall manage change requests through a defined lifecycle (Draft, Submitted, In Review, Approved, Rejected, Implemented, Closed).	UR-05	Must	
SW-13	Implement the ChangeRequest entity and a state machine with allowed transitions.	SR-11	Must	T
SW-14	Implement a change request form (title, rationale, affected objects, priority).	SR-11	Must	D
SR-12	The system shall support a role-based approval workflow.	UR-05	Must	
SW-15	Implement approver assignment by role and approve/reject actions with a single approver.	SR-12	Must	T
SR-13	The system shall allow changes to approved (locked) objects only through an approved change request.	UR-05	Must	
SW-16	Implement an edit guard that rejects changes to locked objects without an approved change request.	SR-13	Must	T
SR-14	The system shall record every change in an audit log.	UR-05, UR-06	Must	
SW-17	Implement an append-only audit log (who, when, old and new value) for all changes.	SR-14	Must	T
UR-06	The user shall be able to freeze, compare and restore baselines and see the full change history.	–	Must	
SR-15	The system shall create immutable baselines and restore them safely.	UR-06	Must	
SW-18	Implement baseline creation as one transaction; reject edits and deletes of baselines.	SR-15	Must	T
SW-19	Implement baseline restore with an automatic safety snapshot.	SR-15	Could	T
SR-16	The system shall compare two baselines, or a baseline and the current state.	UR-06	Should	
SW-20	Implement a diff engine (added, changed, removed) and a comparison UI.	SR-16	Should	T
SR-53	The system shall show the complete change history of any object.	UR-06	Could	
SW-60	Implement a per-object history view built from the audit log.	SR-53	Could	T
UR-07	The user shall be able to use the system from a web browser and discuss requirements with other team members.	–	Must	
SR-17	The system shall be a web application usable in current browsers without installation.	UR-07	Must	
SW-21	Implement a responsive React single-page application served over HTTPS.	SR-17	Must	I
SR-18	The system shall support threaded discussions on objects and change requests.	UR-07	Should	
SW-22	Implement threaded comments with @mentions on objects and change requests.	SR-18	Should	T
SR-19	The system shall notify users of mentions, review requests and suspect links.	UR-07	Could	
SW-23	Implement in-app notifications for mentions, review requests and suspect links.	SR-19	Could	D
SR-39	The system shall support concurrent users on one repository and prevent lost updates.	UR-07	Could	
SW-46	Implement optimistic locking (version field) and a conflict message on concurrent edits.	SR-39	Could	T
SR-40	The system shall let stakeholders validate (sign off) a requirement from the browser.	UR-07	Should	
SW-47	Implement a validation status on objects (Not reviewed, Validated, Rejected) with reviewer name and date.	SR-40	Should	T
UR-08	The user shall be able to exchange requirements with suppliers and partners in Excel and ReqIF formats.	–	Must	
SR-20	The system shall import and export requirements as Excel/CSV.	UR-08	Must	

ID	Requirement	Parent	Priority	Verif.
SW-24	Implement Excel/CSV import (column mapping, preview) and export.	SR-20	Must	T
SR-21	The system shall import and export requirements as ReqIF (.reqif).	UR-08	Should	
SW-25	Implement a ReqIF parser and exporter for a subset (spec objects, spec relations, attributes); .reqifz packages are out of scope.	SR-21	Should	T
UR-09	The user shall be able to link requirements to test cases and record test results.	–	Must	
SR-22	The system shall manage test cases, link them to requirements and derive requirement verification status from test results.	UR-09	Must	
SW-26	Implement test case CRUD, Verifies links and result recording (Pass, Fail, Blocked).	SR-22	Must	T
SW-27	Compute requirement verification status from linked test results.	SR-22	Must	T

3.2.2 Integration, security, AI agent and demo (UR-10 to UR-17)

The AI agent requirements (UR-12 to UR-16) are the focus of the project. Domain profiles cover defense and medical.

ID	Requirement	Parent	Priority	Verif.
UR-10	Other engineering tools shall be able to read and write project data through open interfaces.	–	Should	
SR-23	The system shall provide a documented REST API.	UR-10	Should	
SW-28	Publish a versioned REST API (/api/v1) with JWT authentication and API documentation.	SR-23	Should	I
UR-11	The user shall only access the projects and functions permitted by the assigned role.	–	Must	
SR-25	The system shall authenticate users and enforce role-based permissions.	UR-11	Must	
SW-30	Implement hashed-password login, JWT sessions and a role matrix (Admin, Author, Reviewer, Viewer).	SR-25	Must	T
SR-26	The system shall isolate data per project.	UR-11	Must	
SW-31	Scope every query by project and block cross-project access in middleware.	SR-26	Must	T
SR-48	The system shall support user and group management.	UR-11	Could	
SW-55	Implement user and group administration screens.	SR-48	Could	T
SR-49	The system shall protect communication with TLS and defend against cross-site request forgery.	UR-11	Must	
SW-56	Enforce HTTPS/TLS, CSRF tokens and secure cookie flags.	SR-49	Must	I
UR-12	The AI agent shall separate requirements by domain (defense, medical) and by type.	–	Must	
SR-27	The system shall provide domain profiles (defense, medical) containing the domain rules.	UR-12, UR-14	Must	
SW-32	Implement domain profile configuration (required attributes, forbidden terms, standard references per domain).	SR-27	Must	I
SR-28	The AI agent shall classify each requirement by domain, type and criticality.	UR-12	Must	
SW-33	Implement a classification service (LLM prompt with rule-based fallback) returning domain, type, criticality and confidence.	SR-28	Must	T
SR-33	AI output shall remain a suggestion until a user accepts it, and every AI action shall be logged.	UR-12, UR-13, UR-14, UR-15	Must	
SW-38	Implement a suggestion queue (accept, edit, reject) and log every AI action with the model name.	SR-33	Must	T
UR-13	The AI agent shall check the quality of each requirement.	–	Must	
SR-29	The AI agent shall evaluate requirement quality and give a score with reasons.	UR-13	Must	

ID	Requirement	Parent	Priority	Verif.
SW-34	Implement a quality checker based on requirement-writing best practices: a rule engine (missing “shall”, vague terms, several requirements in one, no verification method) plus an LLM review; score 0 to 100.	SR-29	Must	T
UR-14	The AI agent shall revise requirements to match the rules of the selected domain.	–	Must	
SR-30	The AI agent shall propose a corrected requirement text that follows the selected domain profile.	UR-14	Must	
SW-35	Implement a rewrite service returning proposed text, rationale, standard reference and a diff view.	SR-30	Must	T
UR-15	The AI agent shall draft requirements from documents and suggest traceability links.	–	Must	
SR-31	The AI agent shall draft requirements from uploaded documents.	UR-15	Must	
SW-36	Implement document ingestion (PDF/text), chunking and LLM requirement drafting.	SR-31	Must	D
SR-32	The AI agent shall suggest links between requirements, and between requirements and tests.	UR-15	Should	
SW-37	Implement link suggestion by text similarity with confidence values.	SR-32	Should	T
SR-34	The AI agent shall be able to run on a locally hosted language model without sending data outside the organization.	UR-15	Must	
SW-39	Implement the AI service as a separate component that calls only a configurable local LLM endpoint.	SR-34	Must	I
UR-16	Once the requirements are linked top-down, the system shall analyze the whole chain and report the results.	–	Must	
SR-35	The system shall analyze the complete linked chain (user, system and software requirements and tests) and detect gaps, suspect links, and quality and domain problems.	UR-16	Must	
SW-40	Implement an analysis job that traverses user, system and software requirements and tests and computes coverage, orphans, suspect links, quality and domain compliance.	SR-35	Must	T
SW-41	Implement an AI narrative summary of the findings and recommendations, reviewed by a user.	SR-35	Should	D
SR-36	The system shall present the analysis as a dashboard and an exportable report.	UR-16	Must	
SW-42	Implement the analysis dashboard and export in one format (PDF or Excel).	SR-36	Must	D
UR-17	The team shall be able to demonstrate the application with realistic sample projects.	–	Must	
SR-37	The system shall include sample defense and medical projects and a scripted demo scenario.	UR-17	Must	
SW-43	Implement a seed script for one defense and one medical sample project (at least 30 requirements each, with links and tests).	SR-37	Must	T
SW-44	Write a demo script covering the end-to-end scenario (import, AI classification and quality check, change request, baseline, analysis).	SR-37	Must	D

3.2.3 Clarifications, boundary conditions and exceptional cases

Analysis of the specification exposed the following points. They are recorded here as agreed interpretations; the supervisor can correct them.

#	Point	Analysis decision
C1	Deleting an object	The object is soft-deleted; its ID stays on the blacklist and is never reused (SR-02, SW-03). Links to it become suspect.
C2	Link rules	A link that violates the rule file (e.g. Verifies from a requirement to a requirement) is rejected with the broken rule named (SR-07).
C3	Editing a locked object	Rejected unless an approved change request exists; the attempt is not logged as a change (SR-13, SW-16).
C4	Clearing a suspect link	Needs a reason; reason, user and time go to the audit log (SW-11).
C5	Change request with several approvers	Not supported: a single approver per request (SW-15).
C6	Baseline edit or delete	Always rejected; restore creates an automatic safety snapshot first (SW-18, SW-19).

#	Point	Analysis decision
C7	Concurrent edits	Optimistic locking (version field); the second save gets a conflict message (SW-46, Could).
C8	AI suggestion not reviewed	Stays in the queue; the requirement is never changed automatically (SR-33).
C9	Language model unavailable	AI features fall back to rule-based mode and the user is told it is a lower-capability mode (NFR-06).
C10	Low-confidence AI output	Shown with its confidence value and reasons so that the reviewer can judge it; no automatic acceptance.
C11	Unreadable or scanned PDF	Ingestion reports that no text could be extracted and drafts nothing.
C12	ReqIF files with unsupported content	The subset is imported; unsupported elements (e.g. .reqifz packages) are rejected or skipped with a report (SW-25).
C13	Excel/CSV import errors	A preview shows column mapping and row errors before anything is saved (SW-24).
C14	Cross-project access	Blocked in middleware, including through the REST API (SW-31).
C15	Scope reduction	If time runs short, reduce first the Could requirements (SW-19, SW-23, SW-46, SW-55, SW-60), then ReqIF (SW-25), baseline comparison (SW-20) and the AI narrative summary (SW-41).

3.3 Nonfunctional requirements

The eight nonfunctional requirements are grouped by the quality they describe.

ID	Requirement	Category	Related to
NFR-01	Typical pages shall load in under 2 seconds with 5,000 objects in a project.	Performance	SW-01, SW-02, SW-09
NFR-02	The system shall be started with a single command (container-based deployment).	Deployment / usability	Manufacturability
NFR-03	The user interface shall support Turkish and English.	Usability / localization	SW-21
NFR-04	The system shall work in current versions of Chrome, Firefox and Edge.	Compatibility	SR-17, SW-21
NFR-05	Data shall be stored in a relational database with documented backup and restore.	Reliability / data	SW-01, SW-17
NFR-06	AI features shall degrade gracefully to a rule-based mode when the language model is unavailable.	Reliability / availability	SW-33, SW-34
NFR-07	The system shall run on Windows and Linux hosts (container-based) with a built-in database and no client installation.	Portability	SR-17
NFR-08	Encrypted communication shall use TLS 1.2 or higher.	Security	SR-49, SW-56

Security and privacy needs that appear as functional requirements are SR-25, SR-26 and SR-49 (authentication, role-based permissions, project isolation, TLS, CSRF). Transparency needs of the AI agent are SR-33 (suggestions only, action log with model name) and SR-29 and SW-35 (reasons, confidence, standard reference).

3.4 Pseudo requirements

Pseudo requirements are constraints on the solution that do not describe behaviour. They come from the constraint analysis of the Project Specifications Report.

#	Constraint	Type	Effect on the solution
PR-1	No funding beyond the team’s computers and university resources; the whole stack is free and open source (React front end, relational database, open-weight language model) and no paid cloud or API service is used.	Economic / implementation	Technology choice

#	Constraint	Type	Effect on the solution
PR-2	Total effort of about 157 person-days (about 40 per member) in two semesters; Could requirements stay optional.	Schedule	Scope reduction order (C15)
PR-3	The language model is only used for inference, with an existing, preferably quantized and smaller model, and only when a user starts an AI action.	Environmental	SW-39, NFR-06
PR-4	The AI agent runs only against a locally hosted model; all data stays inside the organization (export-controlled, classified or medical data).	Political / legal	SR-34
PR-5	Personal data must follow KVKK (Law No. 6698) and, for European users, GDPR.	Legal	SR-25, SR-26, SR-49
PR-6	DOORS features are reproduced as concepts only; no IBM code, names or assets are used; the open ReqIF format is used for exchange.	Legal / intellectual property	SR-21
PR-7	Third-party libraries, models and datasets must have licenses compatible with academic and open-source use.	Legal / licensing	All SW
PR-8	TRAQS is a support tool; it is not qualified under DO-178C or IEC 62304, and users stay responsible for verification.	Safety / liability	Document ation
PR-9	The AI service is a separate component with a configurable model endpoint, built, tested and replaceable independently.	Architecture	SW-39
PR-10	Domain profiles and link rules are configuration files, not code.	Sustainability	SW-08, SW-32
PR-11	Data is exchanged only in open formats (Excel/CSV, ReqIF) and through a versioned, documented REST API.	Interoperabilit y	SW-24, SW-25, SW-28
PR-12	Domain rules come from public standard summaries and are cited; texts that are not freely available are not copied. The supervisor validates the rules because the team lacks domain expertise.	Professional / ethical	SW-32
PR-13	Demonstration projects contain only invented or public data.	Privacy	SW-43
PR-14	The team follows the ACM and IEEE codes of ethics; use of AI tools in coding and writing is disclosed according to the course rules.	Ethical / process	–

3.5 System models

3.5.1 Scenarios

The scenarios below describe typical and exceptional use of TRAQS. Names are invented; the sample project is the demonstration project of UR-17.

Scenario 1: Importing documents and drafting requirements (UR-15, UR-12, UR-13)

Ayse, a requirements engineer, opens the defense sample project and uploads a customer specification as PDF. The AI service splits the text into chunks and drafts requirements, each shown in the suggestion queue with a confidence value. Ayse accepts two, edits one and rejects one. For each accepted requirement she starts the classification, which proposes domain, type and criticality, and the quality check, which returns a score of 0 to 100 with reasons (for example: no “shall”, vague term “fast”). Every AI action is logged with the model name. *Exception:* the language model is down; TRAQS shows that the rule-based mode is used and runs only the rule engine.

Scenario 2: Rewriting a requirement for a domain (UR-14)

Mehmet selects a weak medical requirement and chooses the medical domain profile. The AI agent returns a proposed text, the rationale, the standard reference and a diff. Mehmet edits the wording and accepts it; the

requirement changes, the change is written to the audit log (who, when, old and new value) and the links of the requirement are set to suspect.

Scenario 3: Change request on an approved requirement (UR-04, UR-05)

A requirement is approved and locked. Deniz tries to edit it and is told that a change request is needed. She creates a request with title, rationale, affected objects and priority, and submits it. The impact analysis lists downstream system requirements, software requirements and test cases. The reviewer Elif, assigned by role, sets the request In Review and approves it. Deniz edits the object, marks the request Implemented, and a lead closes it. *Exception:* Elif rejects the request; the object stays locked and unchanged.

Scenario 4: Suspect links and impact analysis (UR-04)

When the text of REQ-0042 changes, all links of the object become suspect in the same transaction. The author opens the impact view, reviews each suspect link, and clears each one with a written reason, which is logged. A link remains suspect until somebody clears it.

Scenario 5: Baseline, comparison and restore (UR-06)

Before a customer review, the lead freezes baseline “v1.0” in one transaction. After two weeks of work she compares v1.0 with the current state (added, changed, removed objects) and decides to restore v1.0. TRAQS creates an automatic safety snapshot first, then restores. Any attempt to edit or delete the baseline itself is rejected.

Scenario 6: Exchange with a supplier (UR-08)

The supplier sends requirements as Excel. The author maps the columns in the import preview, fixes two rows with errors, and imports the file. Later the author exports a module as ReqIF for another partner. *Exception:* the partner’s file contains elements outside the supported subset; TRAQS imports the supported part and reports what was skipped.

Scenario 7: Test results and verification status (UR-09)

A test engineer generates a test case and associates the test case with the requirement using the Verifies association and indicates Pass, Fail, or Block. Verification status of the requirement is determined based on the test cases associated with it.

Scenario 8: Chain analysis and report (UR-16)

When the requirements, tests and links are connected from top-down, the lead kicks off the chain analysis process. The task calculates coverage, orphans, suspect links, quality and domain compliance and displays these measures in a dashboard. Then the lead generates the AI narrative summary of the report, reviews and exports it as PDF or Excel file.

Scenario 9: Access control (UR-11)

An administrator forms the group and assigns the Supplier with the Viewer role on the particular project. The Supplier signs in via HTTPS and accesses only that project without any editing options. Requesting access to other projects via the REST API is blocked by the middleware.

3.5.2 Use case model

Four human users and two other systems employ TRAQS. These four users include the Author, Reviewer, Test Engineer, and Viewer; these represent different user roles of one account (Admin, Author, Reviewer, and Viewer in SW-30), where the Test Engineer uses Author privileges on test objects.

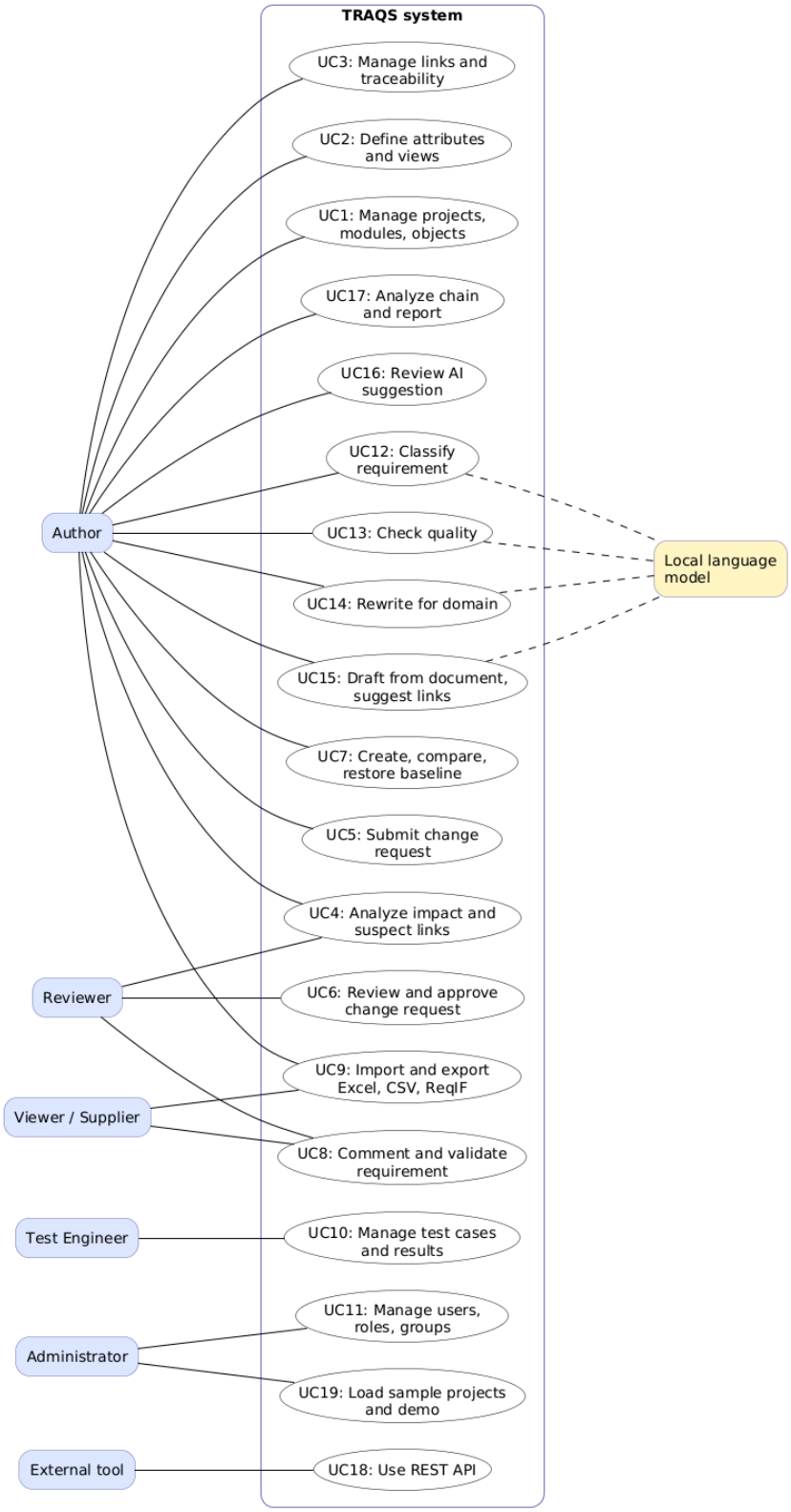


Figure 1. Use case diagram

Use cases for the AI agent (UC12 to UC15) employ the local language model; UC16 (review AI suggestion) is contained in each of them since no output from AI alters any requirement without it (SR-33). UC5 contains UC4 since a change request contains the impact on it.

Use case descriptions

ID	Use case	Actor	Main flow	Alternative / exceptional flow	Requirements
UC1	Manage projects, modules, objects	Author	Create project, module and objects; reorder by drag and drop; the system assigns a unique ID	Delete: ID stays blacklisted	UR-01, SR-01, SR-02
UC2	Define attributes and views	Author	Define a typed attribute; save a personal or shared view with filters, columns, sort	Invalid value type: rejected	UR-02, SR-04, SR-05
UC3	Manage links and traceability	Author	Create a typed link; open matrix or upstream/downstream tree	Link breaks a rule: rejected with the rule	UR-03, SR-06 to SR-08
UC4	Analyze impact and suspect links	Author, Reviewer	Select a changed object; view affected items; clear suspect link with a reason	No reason given: not cleared	UR-04, SR-09, SR-10
UC5	Submit change request	Author	Fill form (title, rationale, affected objects, priority); submit	Draft saved when incomplete	UR-05, SR-11
UC6	Review and approve change request	Reviewer	Open request, review impact, approve or reject	Rejected: objects stay locked	UR-05, SR-11 to SR-14
UC7	Create, compare, restore baseline	Author	Freeze baseline; compare with another baseline or current state; restore with safety snapshot	Edit or delete of baseline: rejected	UR-06, SR-15, SR-16, SR-53
UC8	Comment and validate requirement	Reviewer, Viewer	Post threaded comment with @mention; set validation status	Concurrent edit conflict: message shown	UR-07, SR-18, SR-19, SR-39, SR-40
UC9	Import and export	Author, Viewer	Map columns, preview, import Excel/CSV; export or import ReqIF subset	Row errors or unsupported ReqIF elements: reported	UR-08, SR-20, SR-21
UC10	Manage test cases and results	Test Engineer	Create test case, link Verifies, record Pass, Fail or Blocked; status is computed	Blocked test: status not verified	UR-09, SR-22
UC11	Manage users, roles, groups	Administrator	Create user and group; assign role per project	Cross-project access: blocked	UR-11, SR-25, SR-26, SR-48
UC12	Classify requirement	Author	Start classification; view domain, type, criticality, confidence	Model down: rule-based fallback	UR-12, SR-28
UC13	Check quality	Author	Start check; view score 0 to 100 and reasons	Model down: rules only	UR-13, SR-29
UC14	Rewrite for domain	Author	Choose domain profile; view proposed text, rationale, standard reference, diff	No rule applies: no rewrite proposed	UR-14, SR-27, SR-30
UC15	Draft from document and suggest links	Author	Upload PDF or text; review drafted requirements and link suggestions with confidence	No text found: nothing drafted	UR-15, SR-31, SR-32, SR-34
UC16	Review AI suggestion	Author	Accept, edit or reject from the queue; the action is logged with the model name	Not reviewed: stays in the queue	SR-33
UC17	Analyze chain and report	Author	Run analysis; view dashboard; optional AI summary; export	Unlinked items: reported as orphans	UR-16, SR-35, SR-36
UC18	Use REST API	External tool	Authenticate with JWT; read or write project data	No permission: rejected	UR-10, SR-23
UC19	Load sample projects and demo	Administrator	Run seed script; follow the demo script	–	UR-17, SR-37

3.5.3 Object and class model

This analysis model represents the application domain. Requirements are among many Objects which form the hierarchy and link, test case, change request, and baseline all refer to Objects. The outcome of an AI process would be recorded in Suggestions but can never update a requirement directly. It is a model for analysis class domains, not the design.

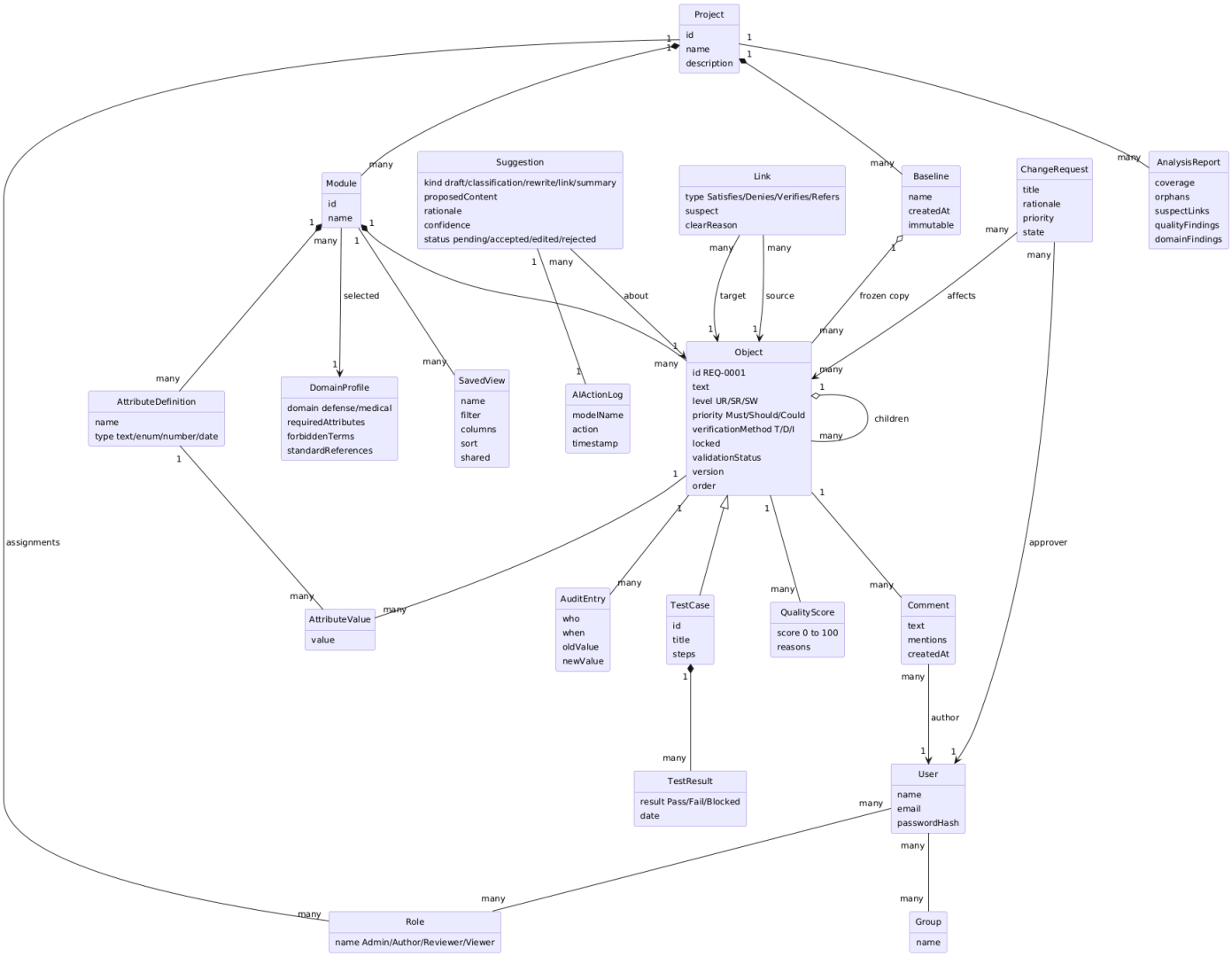


Figure 2. Class diagram (analysis model)

Class responsibilities

Class	Responsibility	Requirements
Project, Module, Object	Container hierarchy; Object has a unique, never-reused ID, level (UR, SR, SW), priority and verification method	SR-01, SR-02
AttributeDefinition, AttributeValue, SavedView	User-defined typed attributes and saved views	SR-04, SR-05
Link	Typed relation between two Objects; has a suspect flag set when the source or target changes; validated against the link rule file	SR-06 to SR-09
TestCase, TestResult	Test objects linked by Verifies; the verification status of a requirement is computed from the results	SR-22
ChangeRequest	Lifecycle Draft, Submitted, In Review, Approved, Rejected, Implemented, Closed; one approver by role	SR-11 to SR-13
Baseline	Immutable frozen copy of the Objects and Links of a project	SR-15, SR-16
AuditEntry	Append-only record of who changed what, when, old and new value	SR-14, SR-53
Comment	Threaded discussion with mentions	SR-18
User, Group, Role	Authentication and role-based permissions per project	SR-25, SR-26, SR-48
DomainProfile	Domain rules (defense, medical) kept in configuration	SR-27
Suggestion, AIActionLog	Every AI output waits as a suggestion until accepted, edited or rejected; every action is logged with the model name	SR-33
QualityScore	Score 0 to 100 with reasons from the rule engine and the language model	SR-29
AnalysisReport	Result of the chain analysis (coverage, orphans, suspect links, quality, domain compliance)	SR-35, SR-36

Object states. An Object is *Unlocked* or *Locked* (approved) and has a validation status (Not reviewed, Validated, Rejected). A Link is *Valid* or *Suspect*. These states are shown in the state diagrams of Section 3.5.4.

3.5.4 Dynamic models

Dynamic models depict interactions between objects of the class models in typical scenarios and how significant objects change their states.

Sequence diagram 1: AI quality check and review of the suggestion (Scenarios 1 and 2; UR-13, SR-29, SR-33, NFR-06)

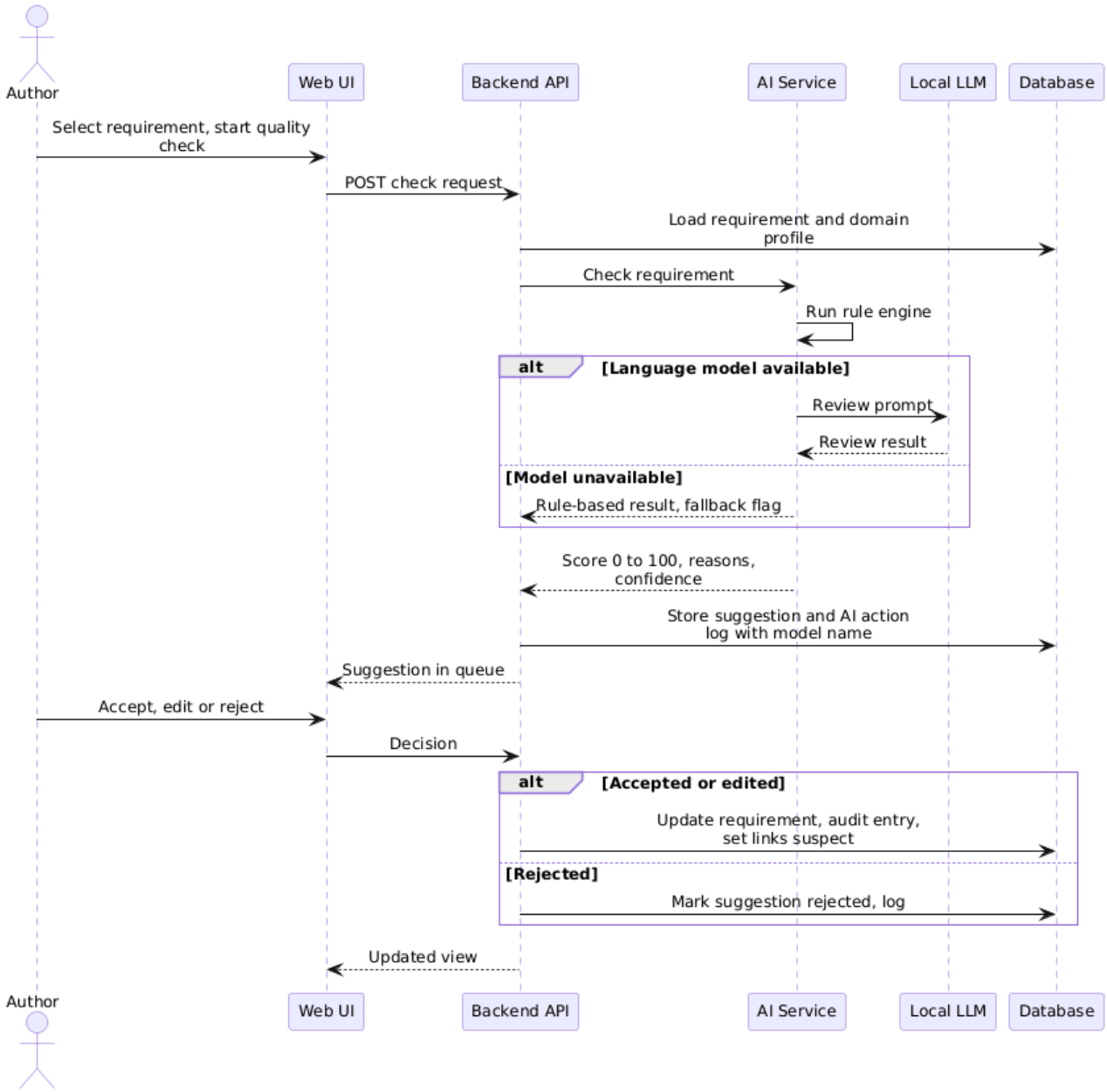


Figure 3. Sequence diagram: AI quality check

Sequence diagram 2: Change request on a locked requirement (Scenario 3; UR-05, SR-11 to SR-14)

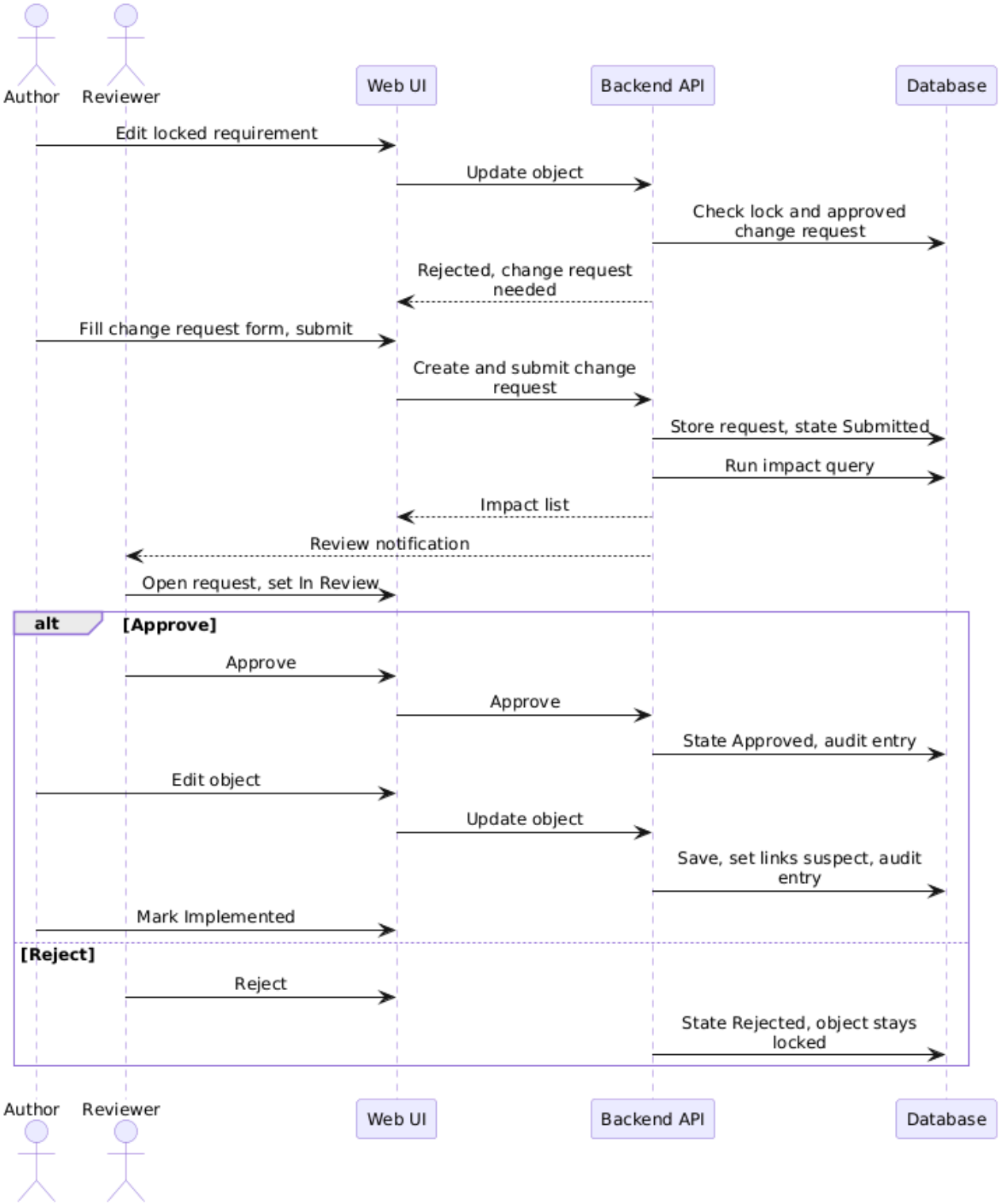


Figure 4. Sequence diagram: change request

State diagram 1: Change request lifecycle (SR-11)

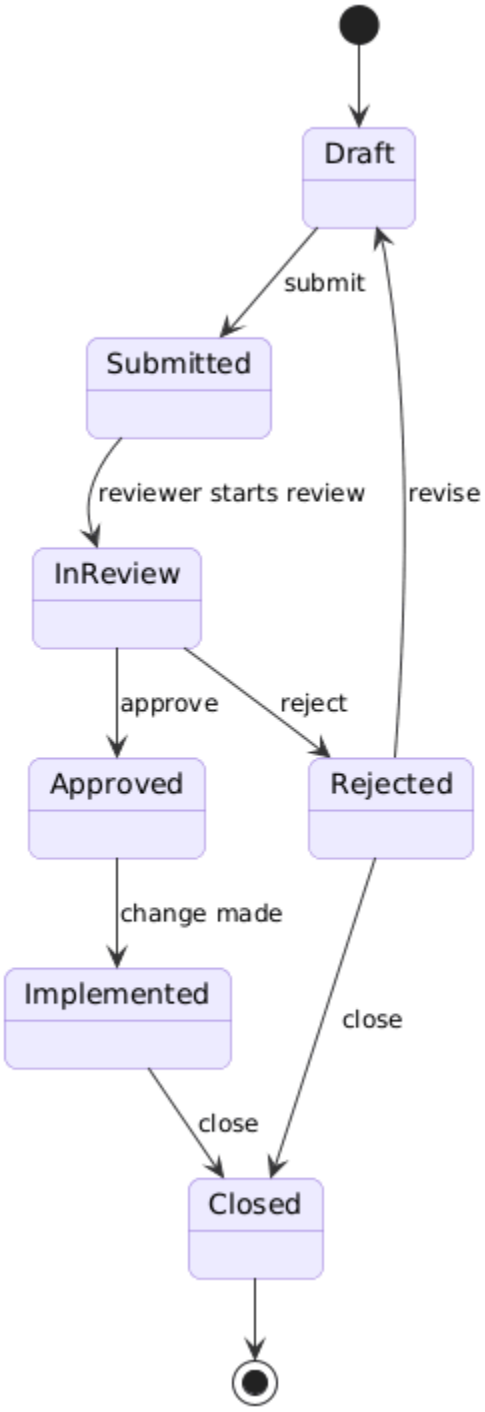


Figure 5. State diagram: change request lifecycle

From “Rejected” back to “Draft (revise)” is one of the analysis assumptions: it can be found in the specification but not all transitions. It has to be verified with the supervisor.

State diagram 2: Link, Object and AI suggestion (SR-09, SR-13, SR-33)

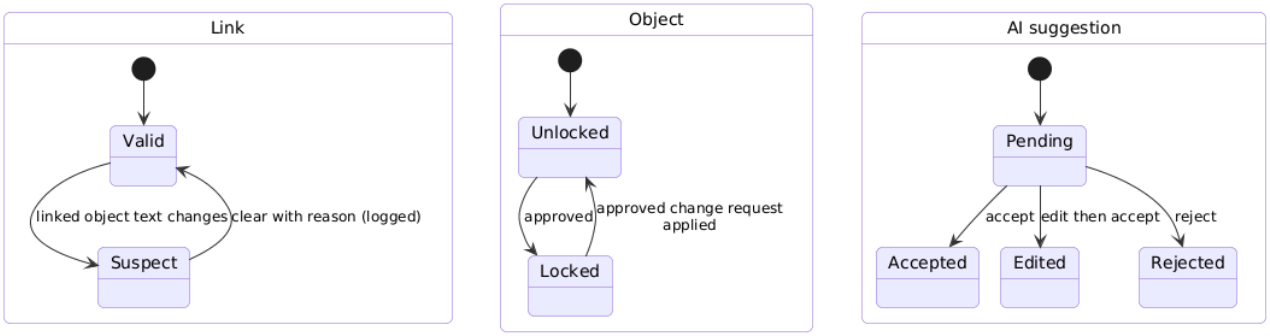


Figure 6. State diagram: link, object, AI suggestion

Activity diagram: Chain analysis and report (Scenario 8; UR-16, SR-35, SR-36)

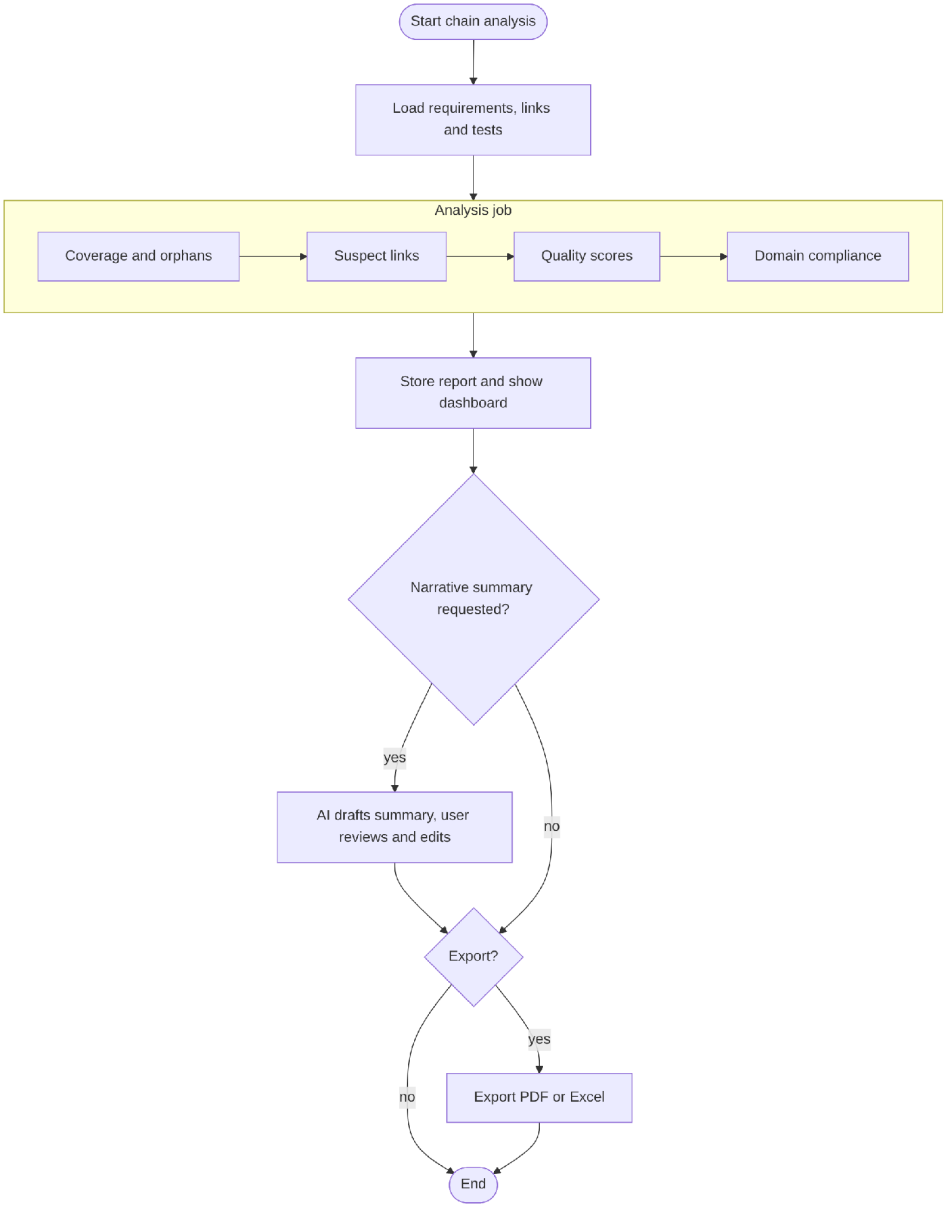


Figure 7. Activity diagram: chain analysis

3.5.5 User interface: navigational paths and screen mock-ups

The user interface is a responsive single-page application (SW-21) that is available in both Turkish and English languages (NFR-03). Every page can be accessed through the top bar (switch projects, search, language selection, user menu) and left navigation of the opened project (SW-20). The availability of features depends on the role of the user (UR-11).

Navigational paths

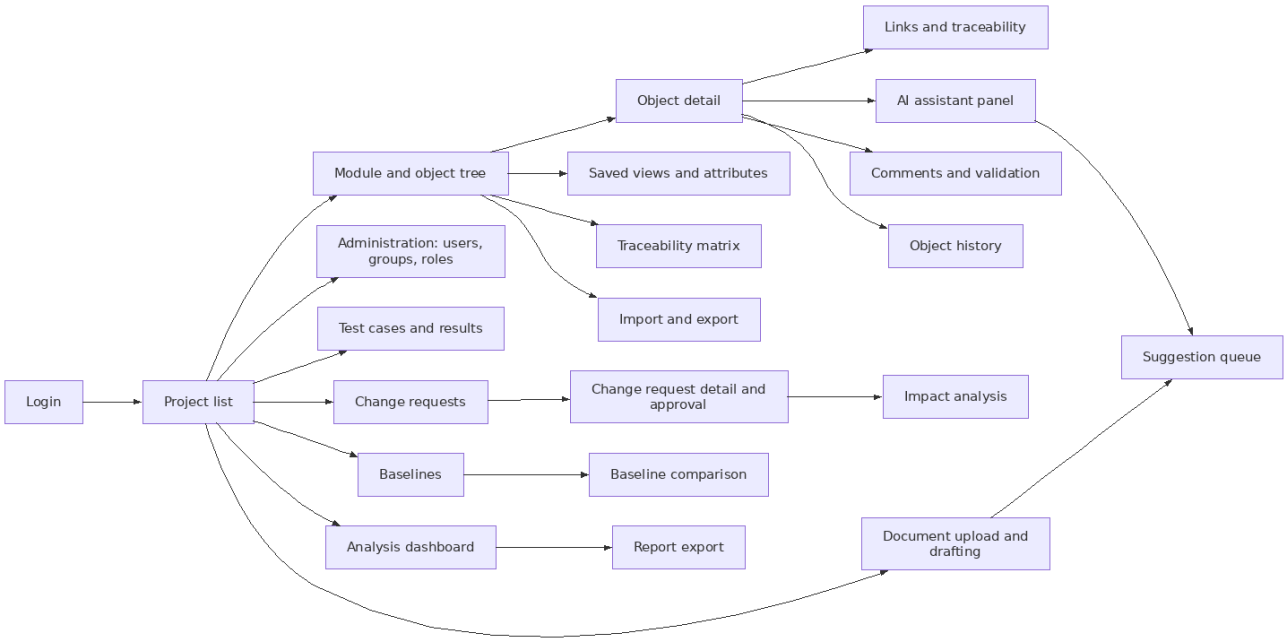


Figure 8. Navigational paths

Screen list

#	Screen	Purpose	Main elements	Requirements
S1	Login	Sign in over HTTPS	User name, password, language switch	SR-25, NFR-03
S2	Project list	Choose or create a project	Project cards, role badge, create button	SR-01, SR-26
S3	Module and object tree	Capture and organize requirements	Tree with expand/collapse and drag and drop, attribute columns, view selector, filter bar, validation and suspect icons	SR-01, SR-02, SR-04, SR-05
S4	Object detail	Edit one requirement	Text, attributes, links, lock state, verification status, comments, history; AI buttons (classify, check quality, rewrite)	SR-06, SR-18, SR-22, SR-40
S5	AI assistant panel and suggestion queue	Review AI output	Score, reasons, confidence, proposed text and diff, standard reference; accept, edit, reject; model name	SR-28 to SR-30, SR-33
S6	Traceability matrix and tree	See coverage and upstream/downstream links	Matrix of requirement by requirement or test, suspect markers, coverage summary	SR-08
S7	Change request detail	Propose, review and approve	Form (title, rationale, affected objects, priority), state, approver actions, impact list	SR-10 to SR-13
S8	Baselines	Freeze, compare and restore	Baseline list, create button, comparison view (added, changed, removed), restore with safety snapshot	SR-15, SR-16
S9	Import and export	Exchange with partners	File selection, column mapping, preview with row errors; ReqIF import and export	SR-20, SR-21
S10	Test cases	Manage tests and results	Test case list, Verifies links, result (Pass, Fail, Blocked)	SR-22
S11	Document drafting	Draft requirements from documents	Upload, progress, drafted requirements with confidence	SR-31, SR-32

#	Screen	Purpose	Main elements	Requirements
S12	Analysis dashboard	See the state of the whole chain	Coverage, orphans, suspect links, quality, domain compliance; AI summary; export	SR-35, SR-36
S13	Administration	Manage access	Users, groups, roles per project	SR-25, SR-48

Screen mock-ups

Below are some examples of the design of the key screens (S3, S4, S5, S7, S12). These are low-fidelity mocks that may be altered while designing.

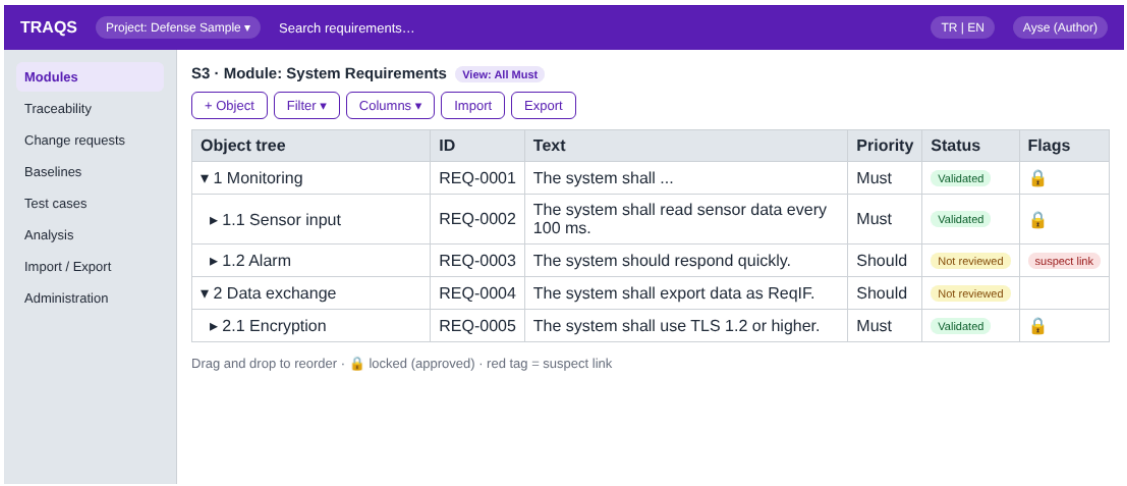


Figure 9. Mock-up S3: module and object tree

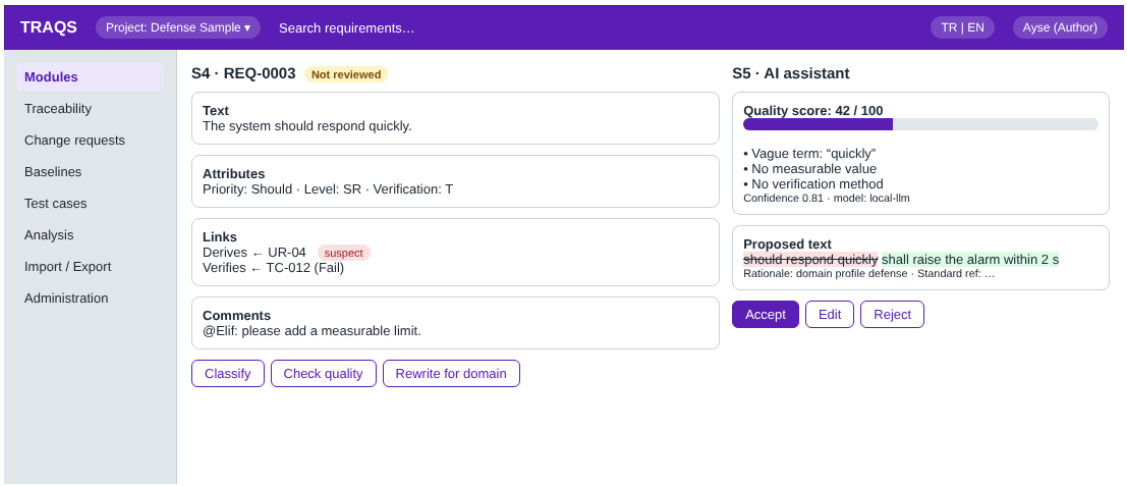


Figure 10. Mock-up S4/S5: object detail and AI assistant

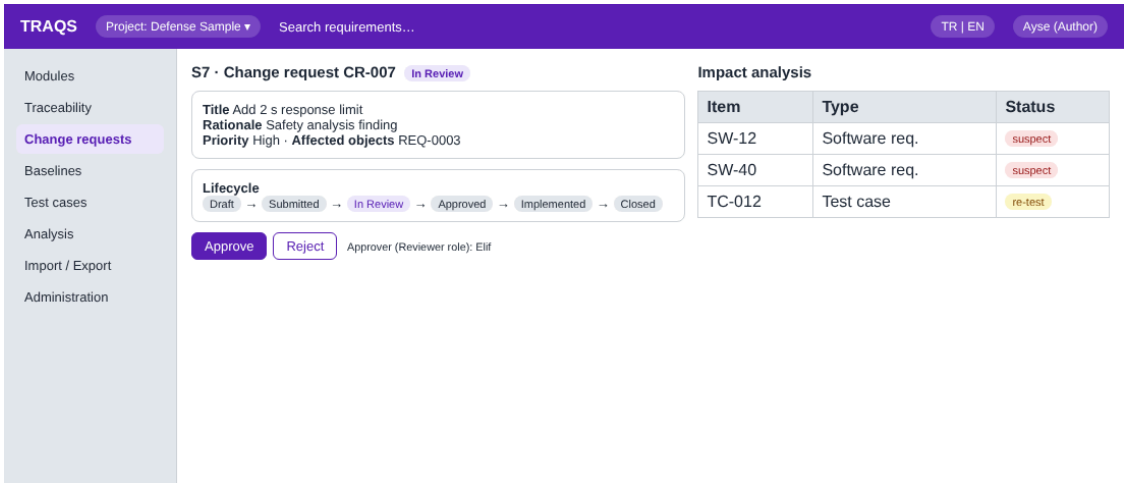


Figure 11. Mock-up S7: change request and impact analysis

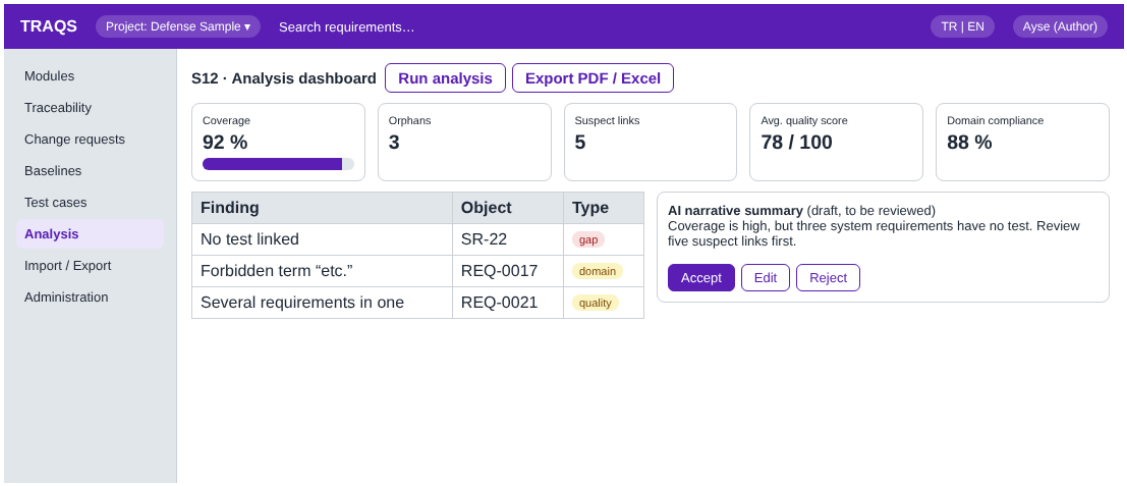


Figure 12. Mock-up S12: analysis dashboard

4. Glossary

Term	Meaning
Audit log	Append-only record of every change (who, when, old and new value)
Baseline	Immutable, frozen state of the requirements of a project at a point in time
Change request (CR)	Formal proposal to change a locked object, with a lifecycle and approval
Chain analysis	Analysis of the whole linked chain of user, system and software requirements and tests
Confidence	Value that shows how sure the AI agent is about a suggestion
Coverage	Share of requirements that are linked to the next level or to a test
CSRF	Cross-site request forgery, an attack that TRAQS defends against
Domain profile	Configuration with the rules of a domain (required attributes, forbidden terms, standard references)
DOORS	IBM DOORS, the requirements management tool used as the reference
Impact analysis	List of items affected by a change to a requirement
JWT	JSON Web Token, used for sessions and API authentication
KVKK / GDPR	Turkish Personal Data Protection Law (No. 6698) / EU General Data Protection Regulation
LLM	Large language model; in TRAQS always hosted locally

Term	Meaning
Locked object	Approved object that can change only through an approved change request
Module	Container of objects inside a project
MoSCoW	Priority scheme: Must, Should, Could
NFR	Non-functional requirement
Object	Basic element of a module (requirement, test case) with a unique, never-reused ID
Orphan	Requirement with no required parent or child link
Quality score	Value from 0 to 100 with reasons, from the rule engine and the LLM review
ReqIF	Requirements Interchange Format of the OMG, used to exchange requirements
REST API	Documented, versioned HTTP interface (/api/v1)
SR / SW / UR	System requirement / software requirement / user requirement
Suggestion	AI output that waits for a user to accept, edit or reject it
Suspect link	Link marked for review because a linked object changed
Traceability matrix	Table that shows links between requirements and tests
Typed link	Link with a type: Satisfies, Derives, Verifies or Refers

5. References

1. Project Specifications Report, CMPE 491 Senior Design Project I, TED University, 2026–2027 Fall (TRAQS group), Oct 20, 2026.
2. Project Proposal, CMPE 491 Senior Design Project I, TED University, 2026–2027 Fall (TRAQS group).
3. B. Bruegge and A. H. Dutoit, *Object-Oriented Software Engineering: Using UML, Patterns, and Java*, 2nd ed., Prentice-Hall, 2004, ISBN 0-13-047110-0.
4. Association for Computing Machinery, “ACM Code of Ethics and Professional Conduct,” 2018. [Online]. Available: https://www.acm.org/code-of-ethics
5. ACM/IEEE Computer Society Joint Task Force on Software Engineering Ethics and Professional Practices, “Software Engineering Code of Ethics and Professional Practice.” [Online]. Available: https://ethics.acm.org/code-of-ethics/software-engineering-code/
6. IEEE, “IEEE Code of Ethics,” IEEE Policies, Section 7.8. [Online]. Available: https://www.ieee.org/about/corporate/governance/p7-8.html
7. OMG, Requirements Interchange Format (ReqIF) specification.
8. Law No. 6698 on the Protection of Personal Data (KVKK, Turkey); Regulation (EU) 2016/679 (GDPR).
9. RTCA DO-178C; IEC 62304 (named in the specification as standards for which TRAQS is not qualified).
10. IBM DOORS product documentation (reference for concepts only).